SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ECA10

MICROPROCESSORS AND MICROCONTROLLERS

MASTER RECORD WITH TEST CASES

Name : Vasantha Nithi D Y

Reg No : 192424137

INDEX

S. No	Name of the Experiment	Page No.
	8085 Microprocessor	
1.	Arithmetic Operations using 8085 Microprocessor	
2.	Ascending and Descending Order of an Unsorted Array using 8085	
3.	ASCII to HEX (Binary) Conversion using 8085 Microprocessor	
4.	Program transferring a block of data between memory locations using 8085	
5.	Perform Masking (AND), Setting (OR) of lower nibbles and One's and Two's complement on given data using 8085 Microprocessor	

1. Arithmetic Operations in 8085 Microprocessors

a. 8-BIT ADDITION WITH CARRY USING DIRECT ADDRESSING

AIM:

To write an assembly language program to add two numbers of 8-bit data stored in memory locations 4200H and 4201H and store the result in 4202H and 4203H with carry using direct addressing.

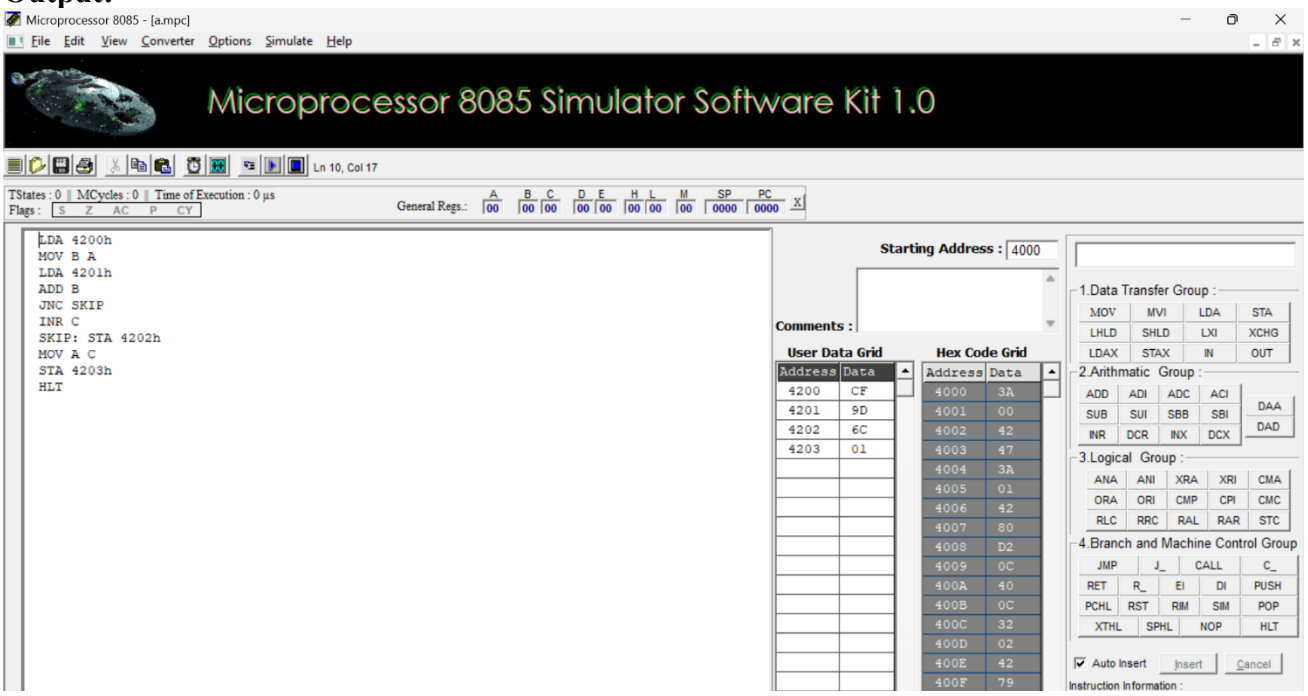
APPARATUS REQUIRED:

1.	8085 microprocessor kit	1
2.	Power card	1
3.	Keyboard	1

ALGORITHM:

1. Load the first data from memory to the accumulator and move it to B register.
2. Load the second data from memory to the accumulator.
3. Add the content of B – register to accumulator
4. If Carry flag = 0 then jump to step 6
5. Increment C register to count the carry
6. Store the sum in memory.
7. Move the carry to accumulator and store in memory.
8. Stop.

Output:



RESULT:

Thus, an assembly language program for addition of two 8-bit numbers with carry was written, executed and Verified the Result successfully using 8085 kit.

b. 8-BIT ADDITION USING INDIRECT ADDRESSING

AIM:

To write an assembly language program to add two numbers of 8-bit data stored in memory locations 4200H and 4201H and store the result in 4202H and 4203H using indirect addressing modes

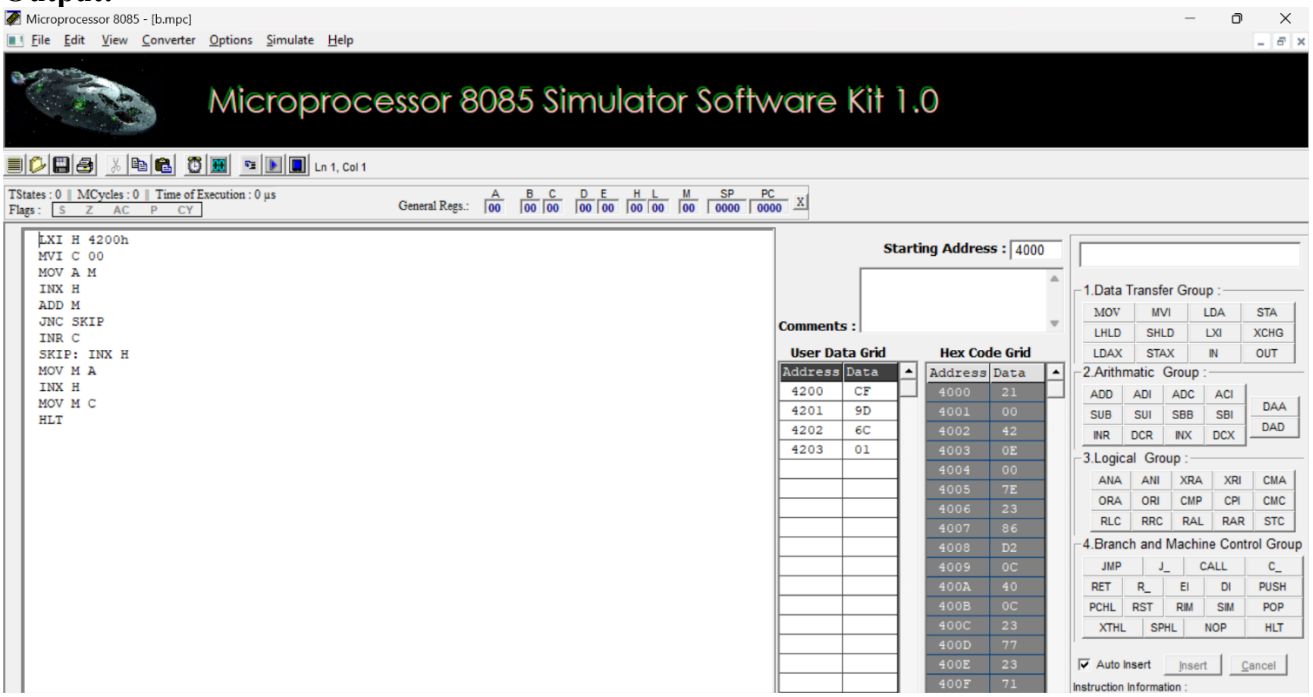
APPARATUS REQUIRED:

1.	8085 microprocessor kit	1
2.	Power card	1
3.	Keyboard	1

ALGORITHM:

1. Load the first data from memory to the accumulator.
2. Add the content of Memory to accumulator
3. If Carry flag = 0 then jump to step 5
4. Increment C register to count the carry
5. Store the sum in memory.
6. Store the carry in memory from C-register.
7. Stop.

Output:



RESULT:

Thus, an assembly language program for addition of two 8-bit numbers with carry was written, executed and Verified the Result successfully using 8085 kit.

c. 8-BIT SUBTRACTION WITHOUT BORROW

AIM:

To write an assembly language program to subtract two numbers of 8-bit data stored in memory 4200H and 4201H. Store the magnitude of the result in 4202H using 8085 Microprocessor.

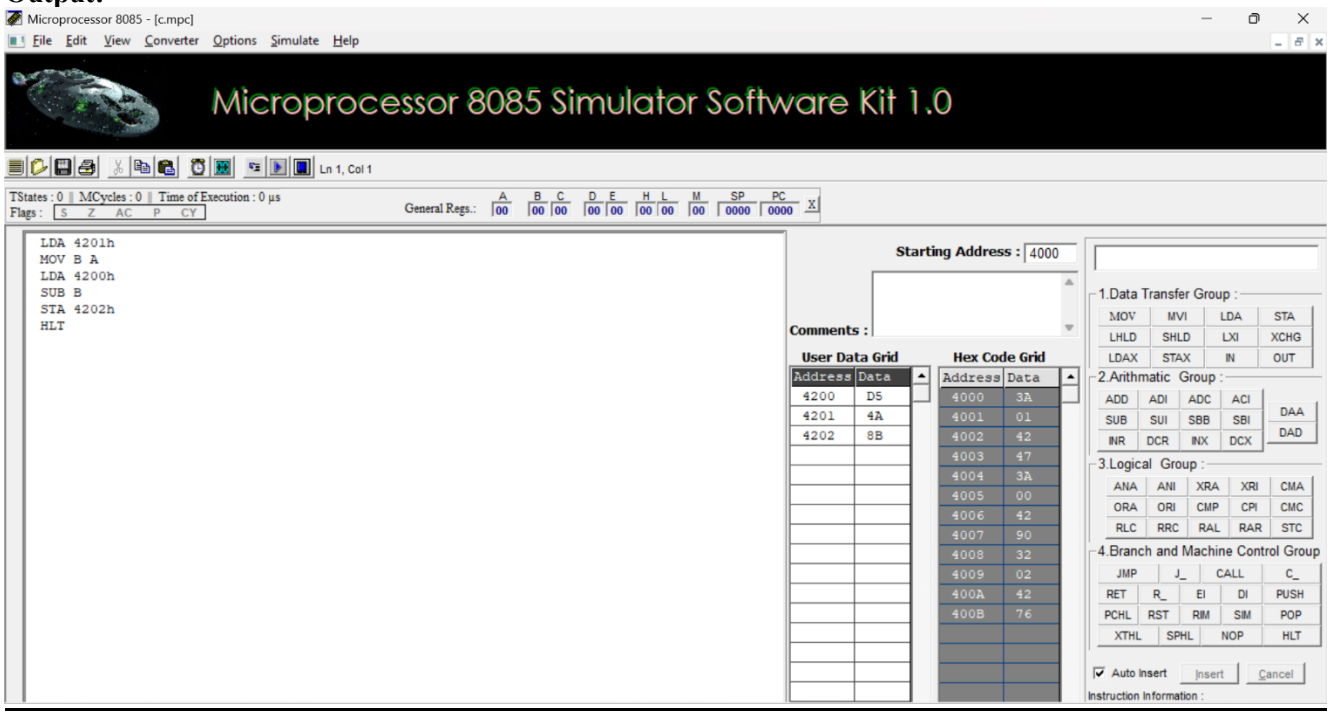
APPARATUS REQUIRED:

1. 8085 microprocessor kit 1
2. Power card 1
3. Keyboard 1

ALGORITHM:

1. Load the subtrahend (the data to be subtracted) from memory to accumulator and move if to B-register.
2. Load the minuend from memory to accumulator.
3. Subtract the content of B-register (subtrahend) from the content of accumulator (minuend).
4. Store the difference in memory.
5. Stop.

Output:



RESULT:

Thus, an assembly language program for subtraction of two 8-bit numbers without borrow was written, executed and Verified the Result successfully using 8085 kit.

d. 8-BIT SUBTRACTION WITH BORROW USING DIRECT ADDRESSING

AIM:

To write an assembly language program to subtract tow numbers of 8-bit data stored in memory locations 4200H and 4201H and store the result in 4202H and 4203H with borrow using direct addressing.

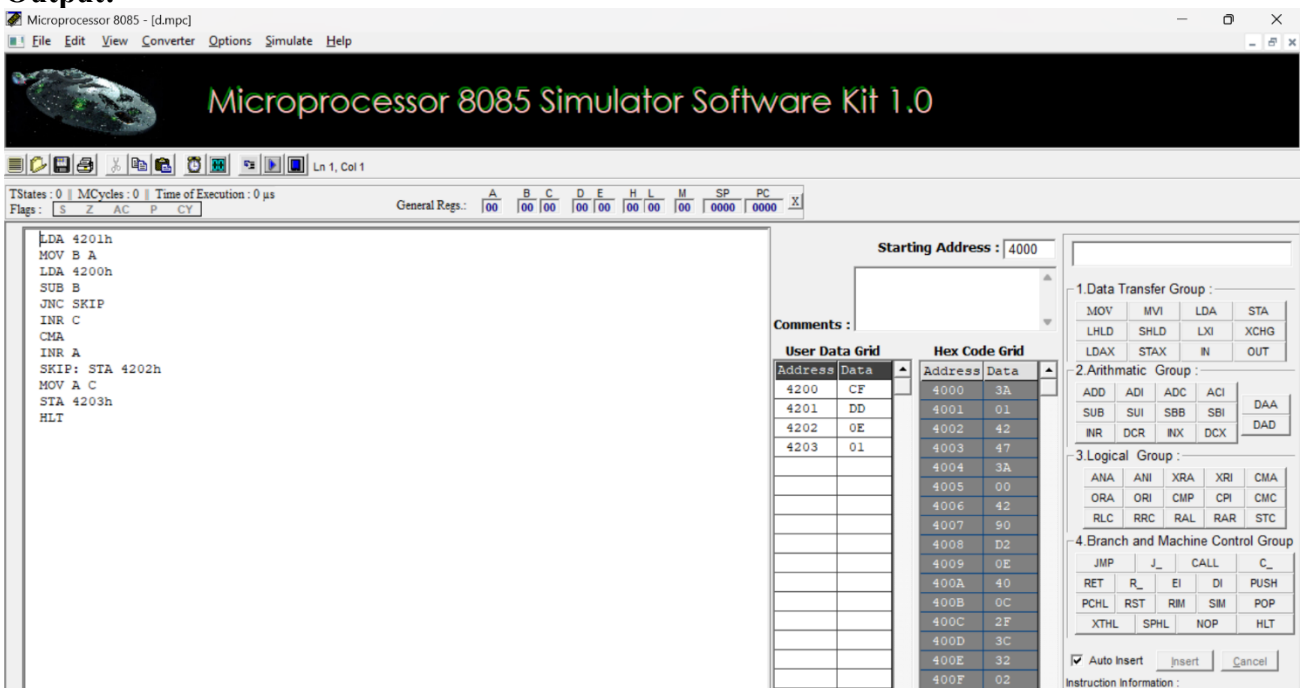
APPARATUS REQUIRED:

1.	8085 microprocessor kit	1
2.	Power card	1
3.	Keyboard	1

ALGORITHM:

1. Load the second data from memory to the accumulator and move it to the B register.
2. Load the first data from memory to the accumulator.
3. Subtract the content of B – register from accumulator
4. If Carry flag = 0 then jump to step 5 & 6
5. Increment C register to count the borrow
6. Take two's complement of the difference
7. Store the Difference in memory.
8. Move the borrow to accumulator and store in memory.
9. Stop.

Output:



RESULT:

Thus, an assembly language program for subtraction of two 8-bit numbers with borrow was written, executed and Verified the Result successfully using 8085 kit.

e. 8-BIT MULTIPLICATION OPERATIONS USING 8085 MICROPROCESSOR

AIM:

To write an assembly language program to multiply two numbers of 8-bit data stored in memory 4200H and 4201H and store the product in 4202H and 4203H.

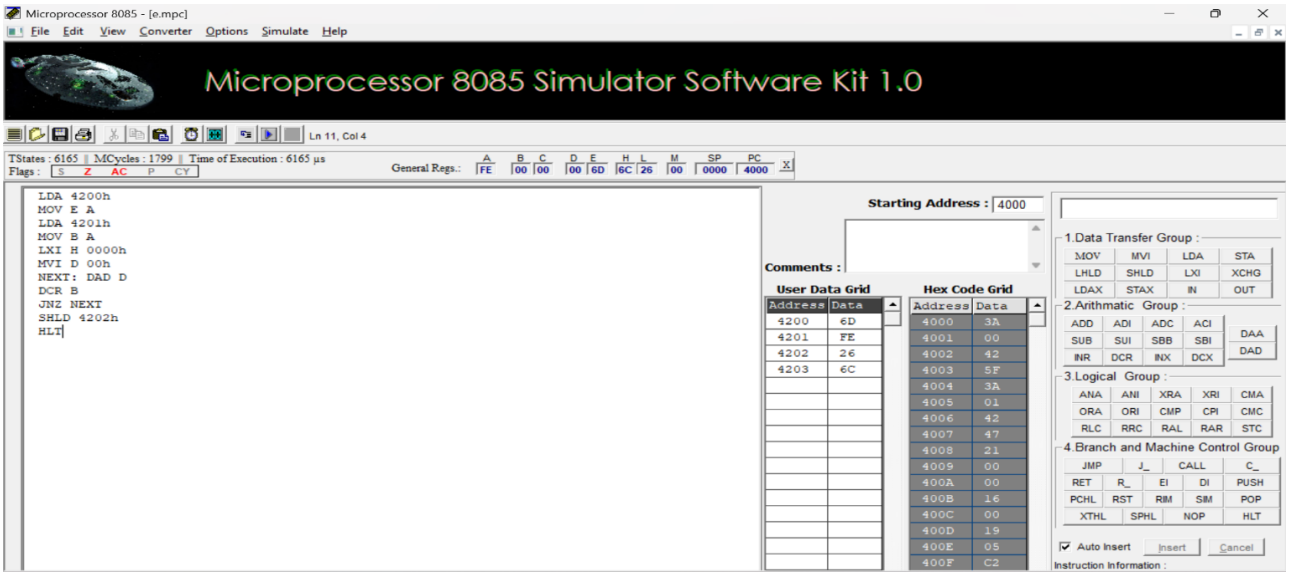
APPARATUS REQUIRED:

1.	8085 microprocessor kit	1
2.	Power card	1
3.	Keyboard	1

ALGORITHM:

- Load the first data in ACC and move to E.
- Load the second data ACC and move to B (count)
- Clear HL pair (Initial sum)
- Clear D for overflow (carry)
- Add the content of DE to HL
- Decrement the count.
- Check whether the count has reached zero.
- Check the zero flag. If ZF = 0, repeat addition or If ZF = 1, go to next step
- Store the content of HL in memory. (Least significant 16 bits of the product)
- Stop.

Output:



RESULT:

Thus, an assembly language program to multiply two numbers of 8-bit data was written, executed and Verified the Result successfully using 8085 kit.

f. 8-BIT DIVISION OPERATIONS USING 8085 MICROPROCESSOR

AIM:

To write an ALP to perform division of two 8 bit numbers Stored in memory location 4200H, 4201 H and Store the remainder in 4202H and the quotient in 4203H.

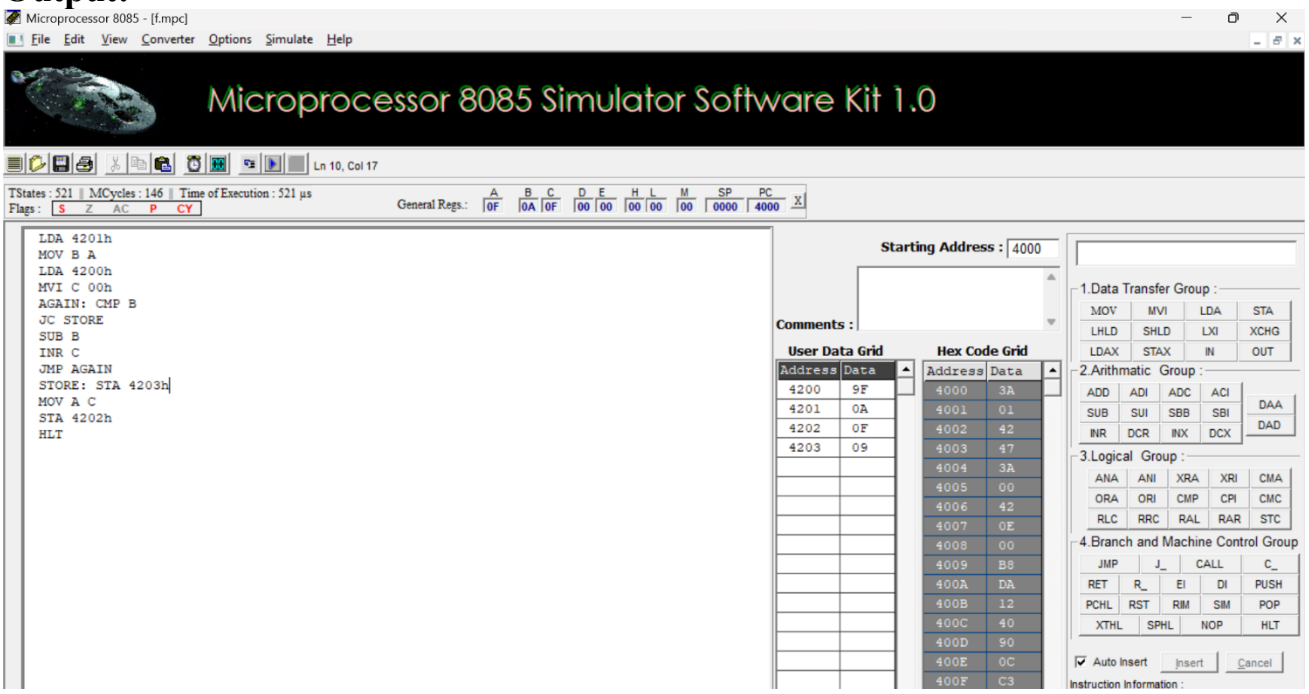
APPARATUS REQUIRED:

1. 8085 microprocessor kit 1
2. Power card 1
3. Keyboard 1

ALGORITHM:

- Load the divisor in accumulator and move if to B-register
- Load the dividend in the accumulator.
- Clear C-register to account for quotient
- Check whether divisor is less than dividend
- If divisor is less than dividend, go to step 8, otherwise go to next step
- Subtract the content of B-register (quotient)
- Increment the content of C-register (quotient)
- Go to step 4
- Store the content of the accumulator (remainder) in memory.
- Move the content of C-register (quotient) to accumulator and store in memory
- Stop.

Output:



Result:

Divide two numbers of 8-bit data was written, executed and Verified the Result successfully using 8085 kits.

Test Cases: Arithmetic Operations of 8085 Microprocessor

Test Case Question 1: These test cases are for the arithmetic operations such as addition and subtraction. Each test case includes the initial values of registers and memory locations, the operation to be performed, and the expected results.

- Initial Register Values:

Accumulator (A) = 35H
B Register (B) = 25H
Operation: What will be the value in the accumulator (A) and the carry flag (CY) after executing the instruction ADD B?

Output:

- The value in the accumulator (A) after the operation will be **5AH**.
- The carry flag (CY) will be **0**.

- Initial Register Values:

Accumulator (A) = 6EH
B Register (B) = 3CH
Operation: What will be the value in the accumulator (A) and the carry flag (CY) after executing the instruction SUB B?

Output:

4. The value in the accumulator (A) after the operation will be **32H**.
5. The carry flag (CY) will be **0** (no borrow).

Test Case Question 2: These test cases are for the arithmetic operations such as multiplication, and division. Each test case includes the initial values of registers and memory locations, the operation to be performed, and the expected results.

(i) Initial Register Values:
Accumulator (A) = 14H
B Register (B) = 0AH
Operation: What will be the values in the accumulator (A) and the B Register (B) after executing the instruction MUL B?

Output:

1. The value in the accumulator (A) after the operation will be **C8H**.
2. The value in the B register (B) will remain **0AH**, as typically multiplication affects only the accumulator.

(ii) Initial Register Values:
Accumulator (A) = 63H (Dividend)
B Register (B) = 09H (Divisor)
Operation: What will be the values in the accumulator (A) and the B Register (B) after executing the instruction DIV B?

Output:

- The value in the accumulator (A) after the operation will be **0BH** (quotient).
- The value in the B register (B) will be **00H** (remainder).

2a.SORT AN ARRAY OF DATA IN ASCENDING ORDER

AIM:

To write an assembly language program to sort an array of data in ascending order. The array is stored in memory starting from 4200H. The first element of the array gives the count value

for the number of elements in the array.

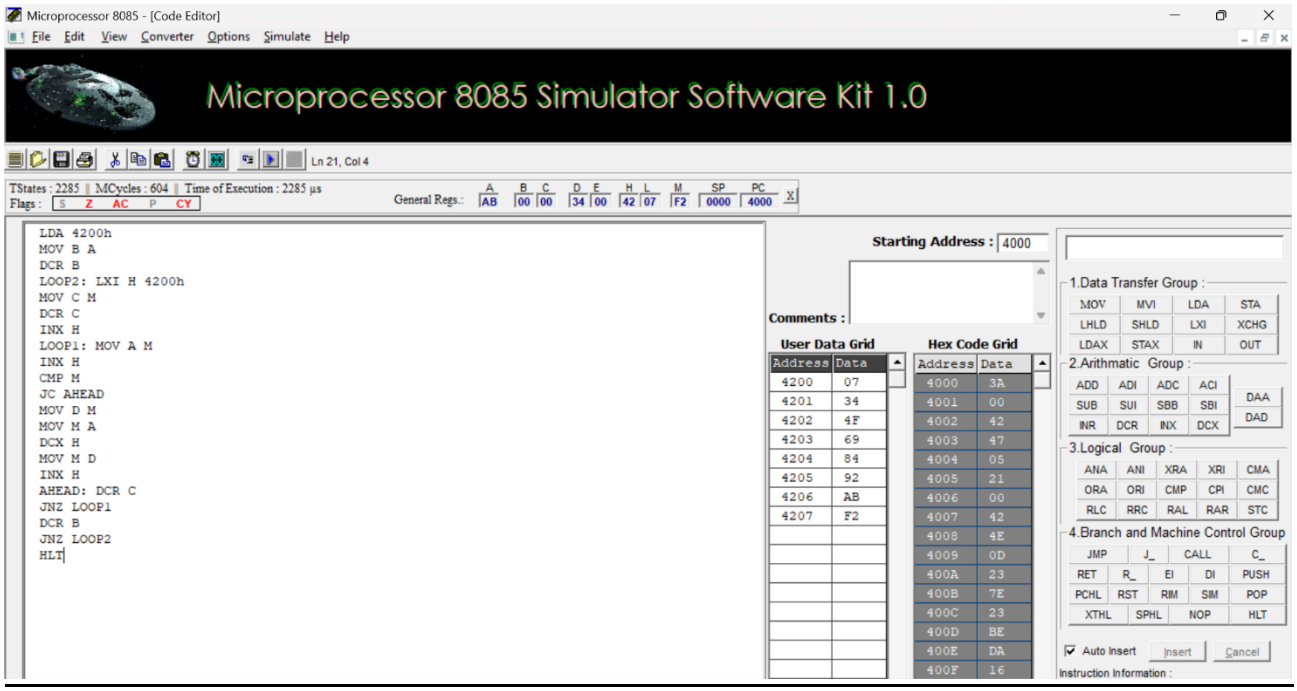
APPARATUS REQUIRED:

1. 8085 microprocessor kit 1
2. Power card 1
3. Keyboard 1

ALGORITHM:

- Load the count value from memory to A-register and save it in B-register
- Decrement B-register (B is a count for (N-1) repetitions)
- Set HL pair as data address pointer
- Set C-register as counter for (N-1) comparisons.
- Load a data of the array in accumulator using the data address pointer
- Increment the HL pair (data address pointer)
- Compare the data pointed by HL wait accumulator
- If carry flag is set (If the content of accumulator is smaller than memory) then go to step 10, otherwise go to next step
- Exchange the content of memory pointed by HL and the accumulator
- Decrement C-register. If zero flag is reset go to step 6 otherwise go to next step
- Decrement B-register. If zero flag is reset go the step 3 otherwise go to next step
- Stop.

Output:



RESULT:

Thus, an assembly language program for sorting in Ascending order of an unsorted array of given 8-bit numbers was written, executed and Verified the Result successfully using 8085 kit.

2b. SORT AN ARRAY OF DATA IN DESCENDING ORDER

AIM:

To write an assembly language program to sort the array of data in descending order. The array is stored in memory stored in memory starting from 4200H. The first element of the array gives the count value for the number of elements in the array.

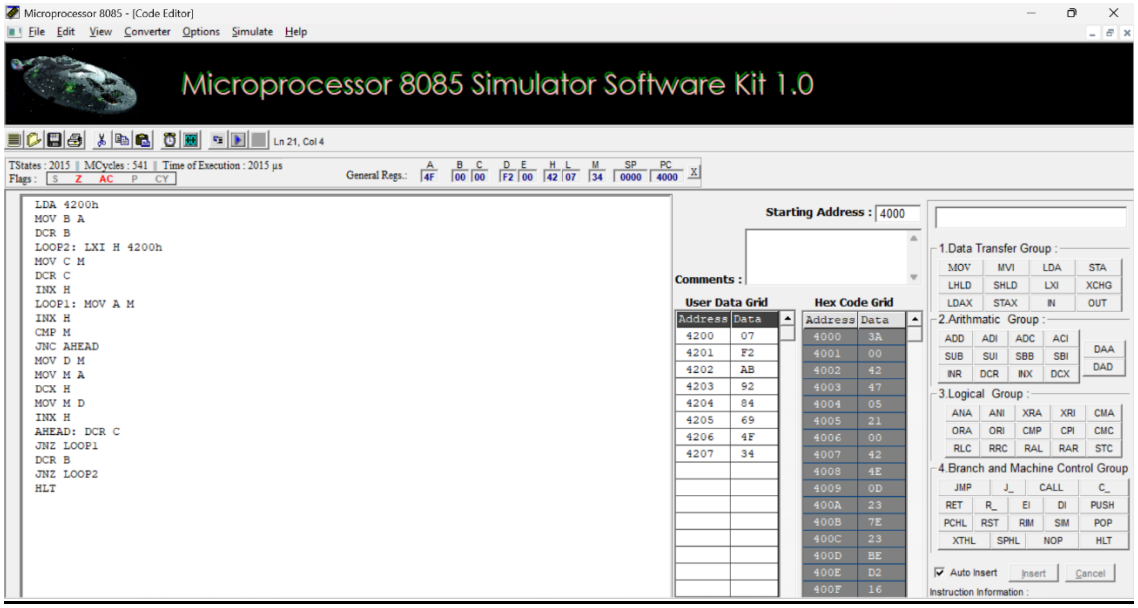
APPARATUS REQUIRED:

1.	8085 microprocessor kit	1
2.	Power card	1
3.	Keyboard	1

ALGORITHM:

- Load the count value from memory to A-register and save it in B-register
- Decrement B-register (B is a count for (N-1) repetitions)
- Set HL pair as data address pointer
- Set C-register as counter for (N-1) comparisons.
- Load a data of the array in accumulator using the data address pointer
- Increment the HL pair (data address pointer)
- Compare the data pointed by HL wait accumulator
- If carry flag is reset (If content of accumulator is larger than memory) then go to step otherwise go to next step
- Exchange the content of memory pointed by HL and the accumulator
- Decrement C-register. If zero flag is reset go to step 6 otherwise go to next step
- Decrement B-register. If zero flag is reset go the step 3 otherwise go to next step
- Stop.

Output:



RESULT:

Thus, an assembly language program for sorting in descending order of an unsorted array of 8-bit numbers was written, executed and Verified the Result successfully using 8085 kit.

Test Cases: Array Operations using 8085 Microprocessor

Test Case Question 1:

a) i)LDA 2050
MOV B, A
LDA 2051
CMP
JNC **
MOV A, B
** STA 2052

In the above Instructions explain the function of compare instruction and what will be the output when you are giving "n" 8- bit numbers.

a ii) Given two 8-bit numbers stored in memory locations 3000H and 3001H as 56H and A3H respectively, write an 8085 assembly program to find the largest number and store it in memory location 3002H
output : The value stored at 3002H should be A3H.

Test Case Question 2:

i) **Test Case Question:** Consider an array with 7 elements 23H, 0AH, 67H, 12H, 4BH, 3CH, 19H stored starting at 2001H, with the size 07H stored at 2000H. Write an 8085 assembly program to sort this array in ascending order.
Output: The sorted array at 2001H should be 0AH, 12H, 19H, 23H, 3CH, 4BH, 67H.

ii) **Test Case Question:Output:** Given an array stored at 2001H with elements 7FH, 80H, 0AH, FFH, and the size 04H stored at 2000H, write an 8085 assembly program to sort this array in ascending order, considering 2's complement representation.
Output: The sorted array at 2001H should be 80H, FFH, 0AH, 7FH.

Test Case Question 3:

i) Test Case Question: Given an array stored in memory starting at location 2001H with 4 elements: 08H, 04H, 0AH, 02H, and the size of the array (04H) stored at 2000H, write an 8085 assembly program to sort this array in descending order.
Output : After execution, the array at 2001H should be sorted as 0AH, 08H, 04H, 02H.

ii) Test Case Question: Given an array stored at 2001H with elements 0FH, F8H, 04H, FFH, and the size 04H stored at 2000H, write an 8085 assembly program to sort this array in descending order, considering 2's complement representation

Output : The sorted array at 2001H should be 0FH, 04H, F8H, FFH.

1. ASCII CODE TO HEX CODE CONVERSION

AIM:

To write an assembly language program to convert an array of ASCII codes to corresponding binary (Hex) value in 8085 Microprocessor

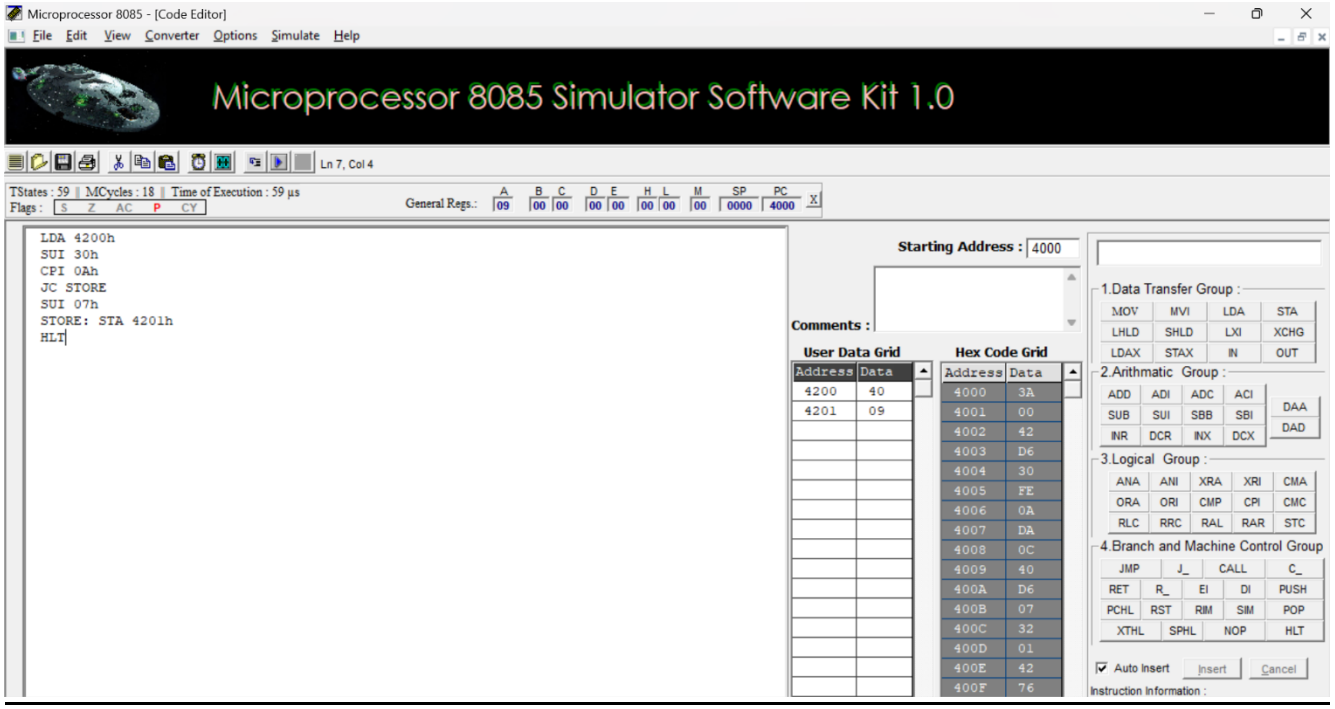
APPARATUS REQUIRED:

1. 8085 microprocessor kit 1
2. Power card 1
3. Keyboard 1

ALGORITHM:

1. Get the ASCII data in A register from 4200H
2. Subtract 30H from A-register
3. Compare the content of A-register with 0AH
4. If CY = 1 go to step 6. If CY = 0, go to next step
5. Subtract 07H from A-register
6. Store the HEX into 4202H
7. Stop the program

Output:



RESULT:

Thus, an assembly language program for converting ASCII to HEX of given 8-bit number was written, executed and Verified the Result successfully using 8085 kit.

Test Cases: Code Conversion using 8085 Microprocessor

Test Case Question :

Convert an ASCII character ('0' to '9') stored in memory into its decimal equivalent and store the result in another memory location.

Test Case Scenarios and Outputs:

Test Case 1:

1. **Input (ASCII)**: The memory location **2500H** contains **0x35** (ASCII for '5').

- The ASCII value (0x35) corresponds to '5'.
- Subtracting 0x30 from 0x35 gives 0x05, which is the decimal value 5.
- The result (0x05) will be stored at memory location **2501H**.

2. **Output**:

- **2501H**: 0x05 (Decimal 5)

Test Case 2:

i) **Input (ASCII)**: The memory location **2500H** contains **0x31** (ASCII for '1').

- The ASCII value (0x31) corresponds to '1'.
- Subtracting 0x30 from 0x31 gives 0x01, which is the decimal value 1.
- The result (0x01) will be stored at memory location **2501H**.

ii) **Output**:

- **2501H**: 0x01 (Decimal 1)

Test Case 3:

- **Input (Invalid ASCII)**: The memory location **2500H** contains **0x41** (ASCII for 'A').

:

  - Since the value is outside the range of ASCII digits (0x30–0x39), the program will jump to the **INVALID** label and halt without modifying memory.
- **Output**:
  - The program halts without storing anything in **2501H**.

Test Case 4:

- **Input (ASCII)**: The memory location **2500H** contains **0x39** (ASCII for '9').

  - The ASCII value (0x39) corresponds to '9'.
  - Subtracting 0x30 from 0x39 gives 0x09, which is the decimal value 9.
  - The result (0x09) will be stored at memory location **2501H**.
- **Output**:
  - **2501H**: 0x09 (Decimal 9)

2. TRANSFERRING A BLOCK OF DATA BETWEEN MEMORY LOCATIONS

AIM:

To write an assembly language program to transfer a block of data from source to destination in 8085 Microprocessor

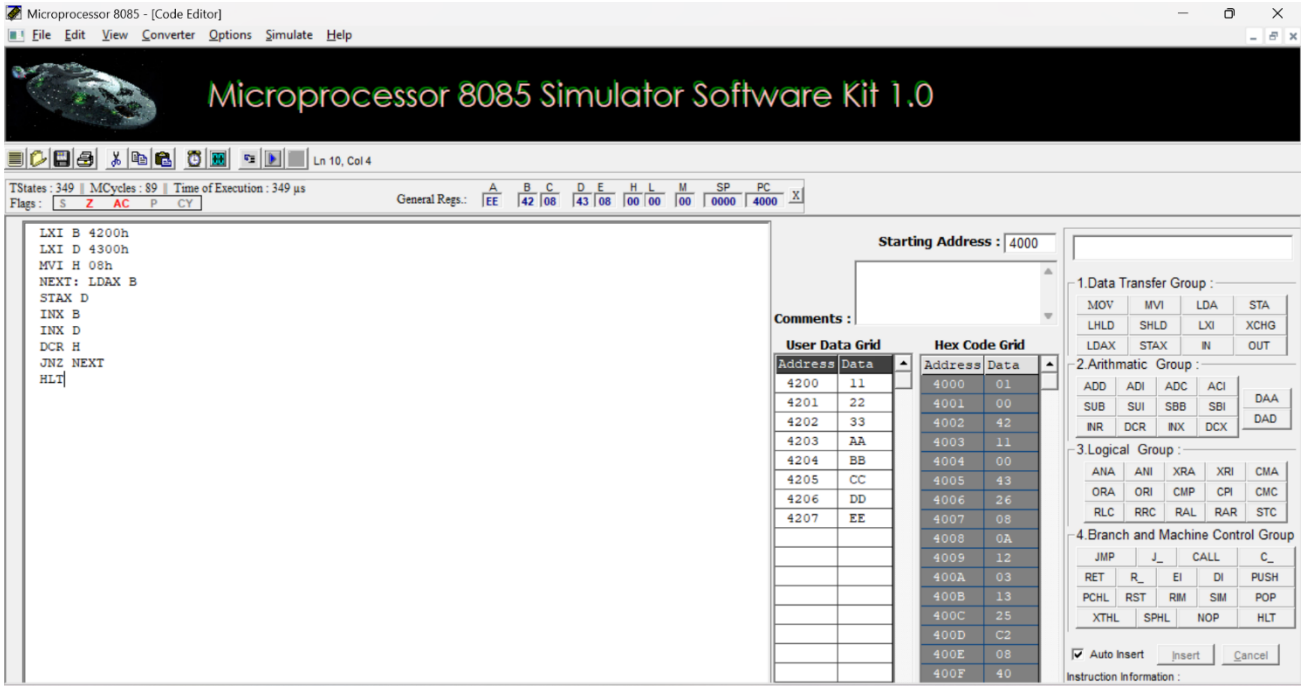
APPARATUS REQUIRED:

1. 8085 microprocessor kit 1
2. Power card 1
3. Keyboard 1

ALGORITHM:

1. Initialize BC pair with 4200H
2. Initialize DE pair with 4300H
3. Initialize H Reg with number of bytes to be transferred
4. Load a Byte from source address and Store the same on Destination address
5. Check all the data bytes transferred if no repeat step 4
6. Stop the program

Output:



Result:

Thus, an assembly language program for Transferring of given 8-bit numbers was written, executed and Verified the Result successfully using 8085 kit.

5a. MASKING AND SETTING OF LOWER NIBBLES ON GIVEN DATA

AIM:

To write and execute an assembly language program for performing Masking, Setting, One's and Two's Complement of given data of 8-bit numbers using 8085 Microprocessor.

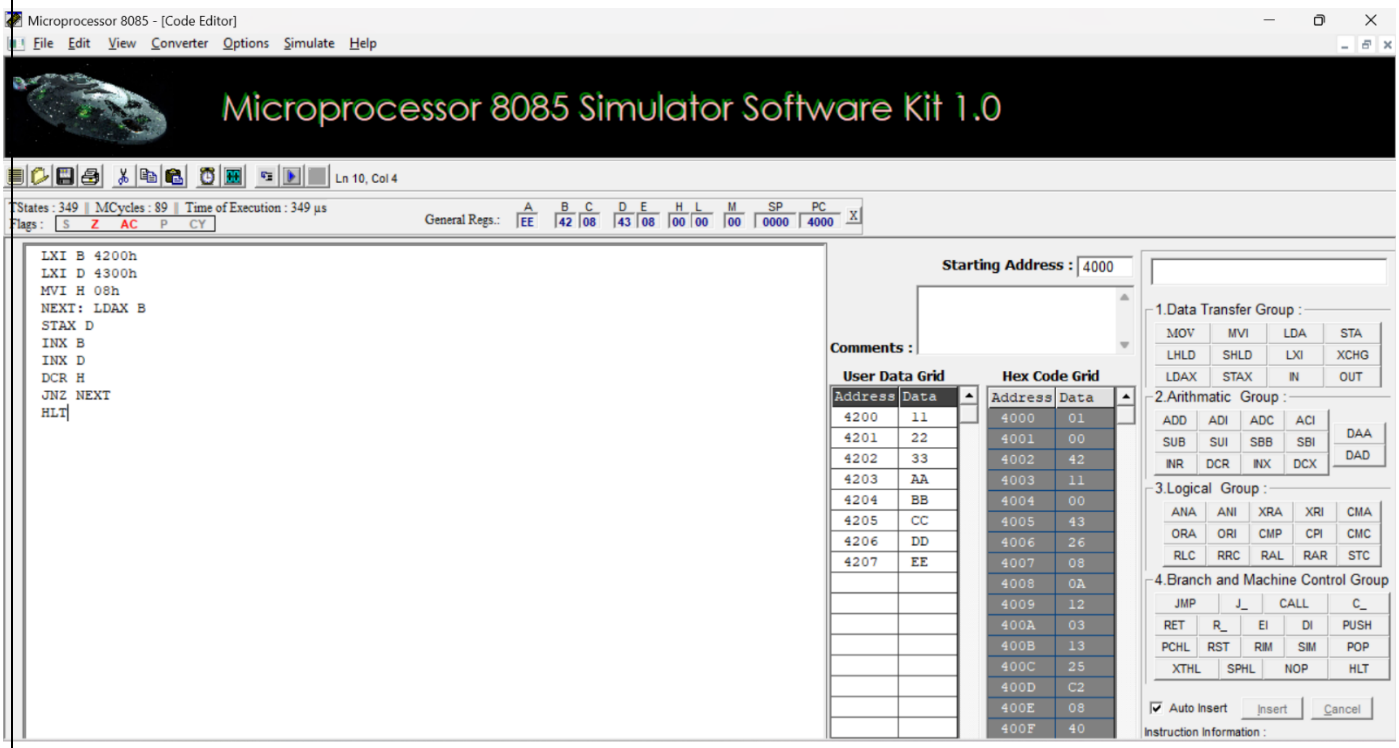
APPARATUS REQUIRED:

8085 microprocessor kit 1
Power card 1
Keyboard 1

MASKING OF BITS ALGORITHM:

1. Load the Data in A-register.
2. Logically AND the content of A with 0FH.
3. Store the result in memory location.
4. Stop the program

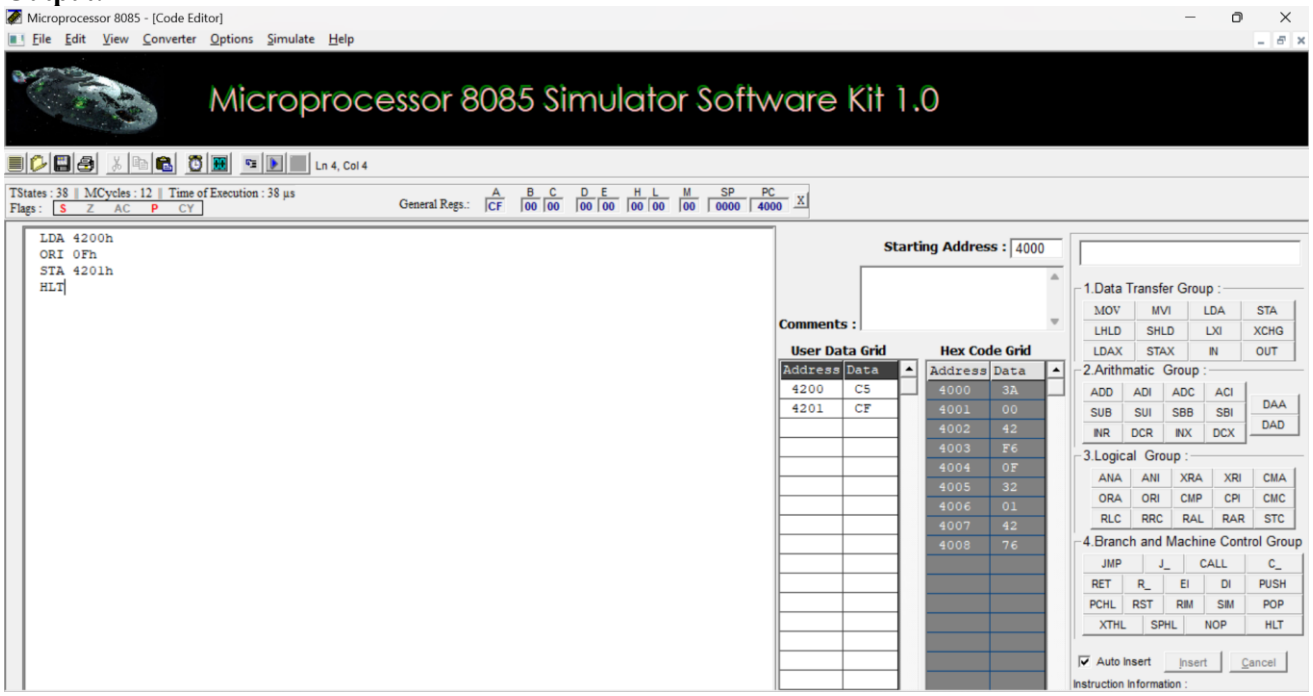
PROGRAM:



SETTING OF BITS ALGORITHM:

1. Load the Data in A-register.
2. Logically ORI the content of A with 0FH..
3. Store the result in memory location.
4. Stop the program

Output:



RESULT:

Thus, an assembly language program for performing logical Masking and Setting of bits were executed using 8085 kits.

5b.ONE'S AND TWO'S COMPLEMENT

AIM:

To write and execute an assembly language program for performing One's and Two's Complement of given 8-bit numbers using 8085 Microprocessor.

APPARATUS:

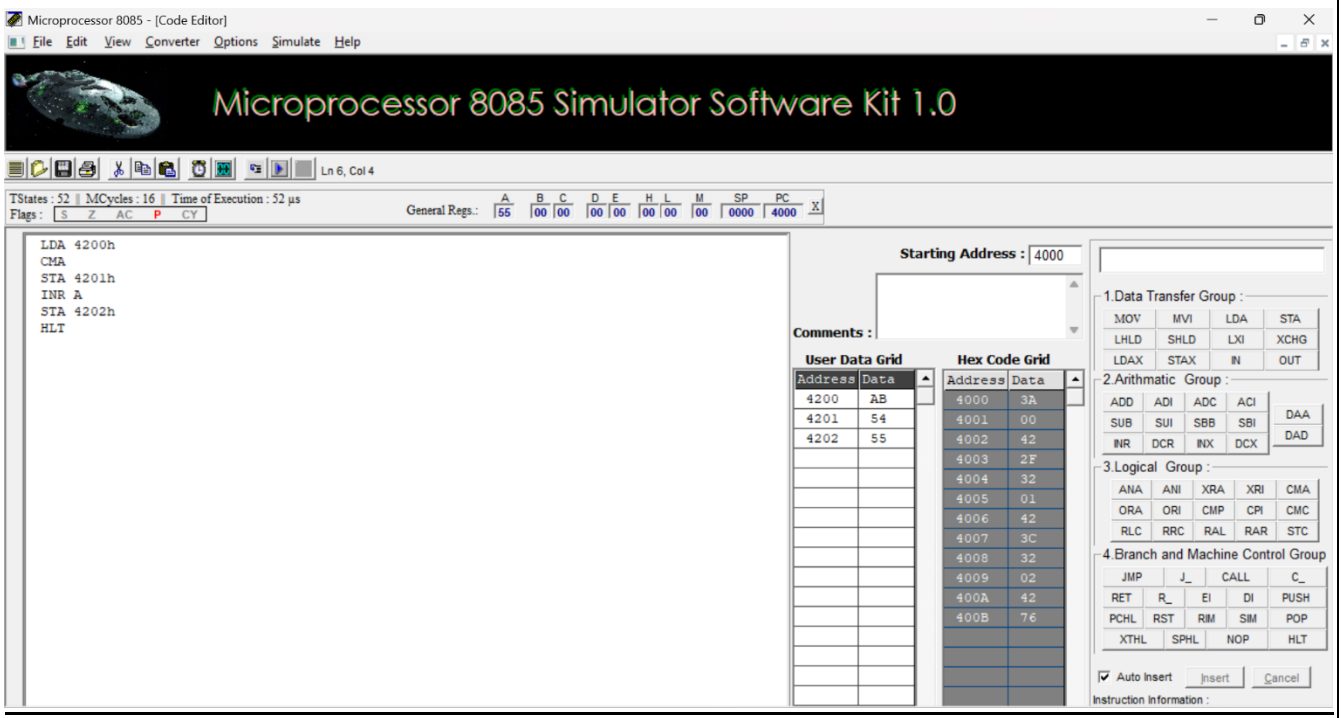
8085 microprocessor kit 1
Power card 1
Keyboard 1

MASKING OF BITS

ALGORITHM:

Load the Data in A-register.
Logically NOT the content of A.
Store the One's complement in memory location.
Increment the content of A.
Store the Two's complement in memory location.
Stop the program

Output:



RESULT:

Thus, an assembly language program for performing One's and Two's Complement of bits were executed using 8085 kit.

Test Cases: Logical operation using 8085 Microprocessor

Test case question 1: Load the accumulator with a random value like 0xB5 and apply ANI 0F.What is the accumulator value when a random bit pattern like 0xB5 is masked with ANI 0F?
10110101
AND 00001111

00000101
OUTPUT:
After executing **ANI 0x0F**, the accumulator will contain **0x05**.

Test case question 2: Load the accumulator with 0x1F and apply the ANI instruction with the immediate data 0x0F.Does the ANI instruction correctly preserve the lower nibble when applied to 0x1F with ANI 0F?
00011111
AND 00001111

00001111
OUTPUT:
After executing ANI 0x0F, the accumulator will contain 0x0F. The lower nibble is correctly preserved.

Test case question 3: Load the accumulator with 0x00 and apply the ORI instruction with the immediate data 0x0F.What is the result in the accumulator when ORI 0F is executed on 0x00?
00000000
OR 00001111

00001111
OUTPUT: After executing ORI 0x0F, the accumulator will contain 0x0F.

Test case question 4: Load the accumulator with 0xFF and perform the one's complement.What happens to 0xFF after performing the one's complement?
~11111111

00000000
OUTPUT: After performing the one's complement on 0xFF, the accumulator will contain 0x00.

Test case question 5:Load the accumulator with 0x80 (representing -128 in signed 8-bit) and perform the one's complement.How does the one's complement work when the accumulator contains a signed negative number like 0x80?
~10000000

01111111
OUTPUT:
After performing the one's complement on **0x80**, the accumulator will contain **0x7F** (which is **127** in signed 8-bit).